Contexte et objectifs

Les systèmes d'autocomplétion reposent sur des statistiques extraites de grandes quantités de texte. Dans ce travail pratique, vous allez implémenter les composantes fondamentales d'un tel système, en mettant l'accent sur le **choix et l'utilisation efficace de structures de données**.

Ce projet n'a pas pour but de reproduire les performances des grands modèles de langage modernes. L'objectif est de comprendre comment des modèles simples basés sur les n-grammes permettent de prédire et compléter du texte de manière efficace.

Objectifs pédagogiques

- Choisir des structures de données adaptées à chaque problème.
- Analyser la complexité temporelle et spatiale d'opérations sur ces structures.
- Implémenter des algorithmes de sélection efficaces (tas, trie).
- Intégrer plusieurs structures au sein d'un modèle de langage fonctionnel.

Architecture du projet

Structure des packages

Package	Contenu
`model/`	Interfaces `NGramModel`, `FrequencyTable`, `TopKStrategy`, `Trie`
`preprocessing/`	`TextTokenizer` — découpage du corpus en phrases/tokens
`gui/`	Interface graphique JavaFX, `Benchmark`, `Grader` (fournis, ne pas modifier)
`student/`	**Vos 4 implémentations à compléter**

Fichiers à modifier

Vous ne devez modifier **que les fichiers suivants**, dans le dossier `src/main/java/ca/udem/ift2015/autocompleter/student/` :

1. `HashFrequencyTable.java` (TODOs 1–6)
2. `HeapTopKStrategy.java` (TODO 7)
3. `PrefixTrie.java` (TODOs 8–10)
4. `KatzBackoffModel.java` (TODOs 11–14)

Tous les autres fichiers sont fournis et ne doivent pas être modifiés.

Interfaces fournies

`FrequencyTable` — associe chaque token à un compteur entier. Fournit `increment()`, `incrementBy()`, `get()`, `total()`, `vocabulary()`, `isEmpty()`.
`TopKStrategy` — sélectionne les k tokens les plus fréquents depuis une `FrequencyTable`. Méthode unique : `topK(FrequencyTable, int)`.
`NGramModel` — contrat du modèle de langage. Fournit `train()`, `predict()`, `topK(int, String...)`, `complete(String, int)`, `unigramCount()`, `bigramCount()`, `trigramCount()`, `frequency(String)`.
`Trie` — contrat de l'arbre préfixe. Fournit `insert()`, `search()`, `complete(String, int)`, `size()`, `nodeCount()`.

Fonctionnement du modèle de langage

Les n-grammes

Un **n-gramme** est une séquence de n tokens consécutifs extraite d'un texte. Le principe fondamental est simple : plus une séquence est fréquente dans un corpus, plus elle a de chances d'apparaître dans de nouveaux textes.

- Un **unigramme** ($n = 1$) est un token isolé. La table des unigrammes recense la fréquence de chaque mot du corpus : combien de fois « le », « chat », « mange » apparaissent-ils ?
- Un **bigramme** ($n = 2$) est une paire de tokens consécutifs (w_{i-1}, w_i). On s'en sert pour estimer la probabilité qu'un mot w_i suive un mot w_{i-1} : par exemple, après « bonjour », le mot « monde » est très probable.
- Un **trigramme** ($n = 3$) est un triplet (w_{i-2}, w_{i-1}, w_i). Il fournit un contexte encore plus précis : après « je vous », on préféra « remercie » à « pizza ».

Entraînement

La phase d'entraînement consiste à parcourir le corpus tokenisé et à compter toutes les occurrences de chaque n-gramme. À la fin, on dispose de trois tables :

Table	Clé	Valeur
`unigrams`	w_i	nombre d'occurrences de w_i
`bigrams`	w_{i-1}	table de fréquences des w_i qui suivent
`trigrams`	w_{i-2} w_{i-1}	table de fréquences des w_i qui suivent

Prédiction : le repli de Katz

Lors de la prédiction, on cherche les k mots les plus probables étant donné un contexte (les derniers mots tapés). Le **repli de Katz** fonctionne par niveaux décroissants de précision :

1. **Niveau trigramme** : si le contexte des deux derniers mots (w_{i-2}, w_{i-1}) est connu dans `trigrams`, on utilise la table de fréquences correspondante — c'est la prédiction la plus précise.
2. **Repli bigramme** : si le trigramme est inconnu mais que le dernier mot w_{i-1} figure dans `bigrams`, on utilise ce niveau.
3. **Repli unigramme** : si aucun contexte n'est disponible, on retourne simplement les mots les plus fréquents du corpus entier.

Ce repli est *strict* : on n'effectue jamais de mélange entre niveaux — dès qu'un niveau est disponible, on l'utilise exclusivement. (Il existe des variantes plus complexes, mais ce n'est pas l'objectif ici.)

Complétion de préfixe

En parallèle du modèle de bigrammes/trigrammes, un **trie** (arbre préfixe) stocke tous les mots du corpus avec leurs fréquences cumulées. Lorsqu'un utilisateur a tapé le début d'un mot, le trie permet de retrouver efficacement tous les mots commençant par ce préfixe et de retourner les k plus fréquents.

Partie I — `HashFrequencyTable` 8 pts

La `HashFrequencyTable` est la structure de données centrale du projet. Elle associe chaque token (mot) à son nombre d'occurrences dans le corpus. Elle est utilisée pour stocker les unigrammes, bigrammes et trigrammes du modèle.

L'implémentation repose sur une `HashMap<String, Integer>` déjà déclarée dans la classe. Un champ `totalCount` doit être maintenu à jour pour que `total()` s'exécute en $\mathcal{O}(1)$.

TODO 1 — `void increment(String token)`

Incrémenter le compteur du token d'une unité. Si le token est absent, l'initialiser à 1. Mettre à jour `totalCount` à chaque appel.

TODO 2 — `int get(String token)`

Retourner le compteur du token, ou **0** si absent.

TODO 3 — `int total()`

Retourner la somme de tous les compteurs. Cette méthode doit s'exécuter en $\mathcal{O}(1)$ — ne pas recalculer à chaque appel, utiliser le champ `totalCount`.

TODO 4 — `Set<String> vocabulary()`

Retourner l'ensemble des tokens ayant été incrémentés au moins une fois.

TODO 5 — `boolean isEmpty()`

Retourner `true` si aucun token n'a encore été incrémenté.

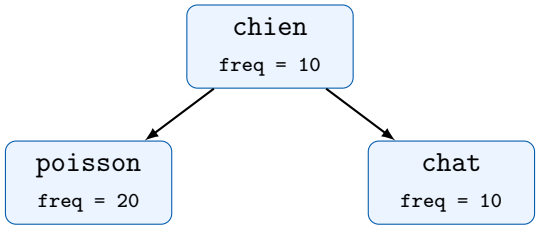
TODO 6 — `void incrementBy(String token, int count)`

Ajouter `count` au compteur du token. Si le token est absent, l'initialiser à `count`. Mettre à jour `totalCount` en conséquence.

Partie II — `HeapTopKStrategy` 10 pts

La `HeapTopKStrategy` implémente la stratégie de sélection des k tokens les plus fréquents. Elle est utilisée par `KatzBackoffModel` pour retourner les meilleures suggestions à chaque niveau du backoff.
L'algorithme repose sur un **min-tas de taille** k : en maintenant le « moins bon » élément au sommet, on peut éliminer efficacement les candidats trop faibles. La complexité cible est $\mathcal{O}(n \log k)$, où n est la taille du vocabulaire.
Le min-tas conserve les k meilleurs candidats vus jusqu'ici. Son sommet est toujours le « moins bon » des k meilleurs éléments retenus, donc celui qui sera évincé si un meilleur candidat arrive. C'est pourquoi le comparateur est l'inverse du tri final : *fréquence croissante*, puis *ordre lexicographique décroissant*.

Min-tas de taille $k = 3$



Remarque

Dans cet exemple, « chien » est le moins bon candidat (fréquence 10, lexicographiquement après « chat »), donc il est au sommet du tas. Si un nouveau candidat arrive avec une fréquence de 15, il remplacera « chien » dans le tas. Si un candidat avec une fréquence de 10 arrive, il ne remplacera « chien » que s'il est lexicographiquement plus petit (ex. « abeille »).

TODO 7 — `List<String> topK(FrequencyTable table, int k)`

Retourne les k tokens les plus fréquents dans `table`, triés par **fréquence décroissante**. En cas d'égalité de fréquence, triés par **ordre lexicographique croissant**.

On a donc le meilleur candidat (fréquence la plus élevée, ou lexicographiquement plus petit en cas d'égalité) en premier dans la liste retournée et le pire candidat (fréquence la plus basse, ou lexicographiquement plus grand en cas d'égalité) en dernier.

Algorithme attendu :

1. Si $k \leq 0$ ou la table est vide, retourner une liste vide.
2. Définir un `Comparator` qui place le « moins bon » au sommet : fréquence *croissante* d'abord, puis ordre lex. *décroissant* (ainsi l'élément évincé est le moins fréquent, et en cas d'égalité, le dernier lex.).
3. Parcourir `table.vocabulary()` :
   - ajouter chaque token au tas
   - si `heap.size() > k`, retirer le sommet (`heap.poll()`).
4. Lorsqu'on a traité tous les tokens, le tas contient les k meilleurs candidats, mais dans l'ordre partiel inverse (le moins bon au sommet). Vous devez retrourner une liste triée par fréquence décroissante, puis ordre lexicographique croissant.

Remarque

Concernant le point 4, si on extrait les éléments d'un coups *exemple :* `new ArrayList<>(heap)`, l'ordre n'est pas garanti il faudra donc trier la liste avec le même comparateur avant de l'inverser. Si on les extrait un par un avec `heap.poll()`, on les obtiendra déjà dans l'ordre inverse, donc pas besoin de trier avant d'inverser. De plus si on utilise une LinkedList pour stocker les éléments extraits du tas, on peut simplement faire `addFirst()` pour construire efficacement la liste finale dans le bon ordre.

Partie III — `PrefixTrie` 14 pts

Le `PrefixTrie` permet la complétion de mots à partir d'un préfixe. Il stocke tous les mots du corpus avec leurs fréquences cumulées. Chaque nœud interne (`TrieNode`) contient :

- `HashMap<Character, TrieNode> children` — les nœuds fils,
- `boolean isEndOfWord` — marque si ce nœud termine un mot,
- `int frequency` — fréquence cumulée du mot (si `isEndOfWord`).

Deux compteurs sont maintenus à la racine :
`size` (nombre de mots distincts) et `nodeCount` (nombre de nœuds créés, hors racine).

TODO 8 — `void insert(String word, int frequency)`

Insére un mot dans le trie et ajoute `frequency` à sa fréquence cumulée. Si le mot est absent, crée les nœuds manquants

1. Partir de `root` et descendre caractère par caractère.
2. Pour chaque caractère, si le nœud fils est absent, le créer et incrémenter `nodeCount`.
3. Lorqu'on atteint le dernier caractère du mot, on est au nœud terminal :
   - ajouter `frequency` à `node.frequency`.
   - Si le mot n'était pas présent (`isEndOfWord` = `false`), le marquer comme terminal (`isEndOfWord = true`) et incrémenter `size`.

Remarque

L'interface `Trie` fournit une surcharge `insert(String word)` sans le paramètre `frequency`, (`insert(word)` équivaut à faire `insert(word, 1)`). C'est cette version que `KatzBackoffModel` utilise lors de l'entraînement. vous n'avez donc pas à implémenter cette surcharge vous-même.

TODO 9 — `int search(String word)`

Retourner la fréquence cumulée du mot, ou **0** s'il est absent.

1. Partir de `root` et descendre caractère par caractère ; retourner **0** dès qu'un nœud enfant est manquant.
2. Au nœud terminal, retourner `node.frequency` si `isEndOfWord` est vrai, **0** sinon.

TODO 10 — `List<String> complete(String prefix, int k)`

Retourner les k mots commençant par `prefix`, triés par fréquence décroissante (à égalité : ordre lexicographique croissant).

1. Si $k \leq 0$, retourner une liste vide.
2. Descendre jusqu'au nœud correspondant au préfixe ; si absent, retourner une liste vide.
3. Appeler `dfs(node, new StringBuilder(prefix))` pour obtenir une `FrequencyTable` de tous les mots du sous-trie.
4. Utiliser `new HeapTopKStrategy().topK(table, k)` pour obtenir les k mots les plus fréquents du sous-trie et les retourner.

Remarque

La méthode `dfs` est fournie et ne doit pas être modifiée. Elle retourne une `FrequencyTable` contenant tous les mots du sous-trie avec leurs fréquences, ce qui permet de réutiliser directement `HeapTopKStrategy`.

Partie IV — `KatzBackoffModel` 18 pts

Le `KatzBackoffModel` est le modèle de langage principal. Il implémente le **repli de Katz** (*Katz Backoff*) : si un contexte trigramme est connu, les suggestions proviennent exclusivement de ce niveau ; sinon, on « replie » vers le bigramme, puis vers l'unigramme.

Il utilise trois `FrequencyTable` internes :

- `unigrams` — fréquence globale de chaque token,
- `bigrams` — `Map<String, FrequencyTable>` : pour chaque mot précédent w_{i-1}, table de fréquences de w_i,
- `trigrams` — `Map<String, FrequencyTable>` : pour chaque paire "$\texttt{w}_{\texttt{i}-2}$ $\texttt{w}_{\texttt{i}-1}$", table de fréquences de w_i.

Il maintient aussi un `PrefixTrie` pour la complétion de mots.

TODO 11 — `void train(List<List<String» sentences)`

Entraîner le modèle sur une liste de phrases tokenisées.
`sentences` est une liste de phrases, où chaque phrase est une liste de tokens. chaque token est un mot du corpus. Le traitement du texte est déjà fait pour vous, tout les mots sont en minuscules et les phrases sont correctement segmentées.
Pour chaque phrase et pour chaque token/mot w_i de la phrase, effectuer les opérations suivantes :

- Incrémenter `unigrams` pour w_i.
- Insérer w_i dans `trie` via `trie.insert(`w_i`)`.
- Si $i \geq 1$: incrémenter `bigrams[`w_{i-1}`]` pour w_i.
- Si $i \geq 2$: incrémenter `trigrams[`$w_{i-2}\sqcup w_{i-1}$`]` pour w_i (clé = `"w_i-2 w_i-1"`).

Remarque

les clés pour les trigrammes doivent être construite en concaténant les deux mots précédents avec un espace entre eux, par exemple : "le chat" pour les mots w_{i-2} = `"le"` et w_{i-1} = `"chat"`.

Donc pour la phrase "le chat mange le chat", on aura :

Unigrammes	Bigrams	Trigrams
le : 2	le → chat : 2	le chat → mange : 1
chat : 2	chat → mange : 1	chat mange → le : 1
mange : 1	mange → le : 1	mange le → chat : 1

TODO 12 — `List<String> topK(int k, String... context)`

Retourner les k suggestions les plus probables selon le contexte.
Repli strict de Katz (pas de mélange inter-niveaux) :

1. Si `context.length >= 2` et que le trigramme `"context[n-2] context[n-1]"` est connu dans `trigrams` → retourner `strategy.topK(trigrams.get(key), k)`.
2. Sinon, si `context.length >= 1` et que le bigramme `context[n-1]` est connu dans `bigrams` → retourner `strategy.topK(bigrams.get(prev), k)`.
3. Sinon → retourner `strategy.topK(unigrams, k)`.

Note : si $k \leq 0$ ou si le modèle est vide (`unigrams.isEmpty()`), retourner une liste vide.

TODO 13 — `String predict(String... context)`

Retourner le mot suivant le plus probable étant donné le contexte, ou `null` si aucune prédiction n'est possible.
Paramètre : `context` — les derniers mots déjà tapés (varargs, peut être vide). Même sémantique que dans TODO 12 : zéro, un ou deux mots de contexte déclenchent le niveau de repli approprié.
Déléguer à `topK(1, context)` et retourner le premier élément de la liste, ou `null` si la liste est vide.

TODO 14 — `List<String> complete(String prefix, int k)`

Retourner les k mots du vocabulaire commençant par `prefix`, triés par fréquence décroissante (à égalité : ordre lexicographique croissant).
Paramètres :

— `prefix` — début de mot saisi par l'utilisateur (ex. `"cha"` → peut retourner `["chat", "chaise", ...]`). Une chaîne vide retourne les k mots les plus fréquents du corpus entier.

— `k` — nombre maximum de suggestions à retourner.

Déléguer à `trie.complete(prefix, k)`.

Interface graphique

L'interface graphique est lancée avec :

```
mvn javafx:run
```

Elle comporte quatre onglets :

Corpus Sélectionnez un ou plusieurs fichiers `.txt` dans `src/main/resources/corpus/`, puis cliquez sur « Entraîner la sélection ». Les statistiques (nombre d'unigrammes, bigrammes, trigrammes) s'affichent après l'entraînement.

Démo Testez l'autocomplétion en temps réel. La zone supérieure affiche le texte composé, les trois boutons de suggestion proposent les mots les plus probables selon le contexte. Un tableau détaille les top-5 de chaque niveau (trigramme, bigramme, unigramme, trie).

Benchmark Mesure le temps d'entraînement, les temps de requête `topK` et `complete`, ainsi que la mémoire utilisée, pour un ou plusieurs fichiers corpus.

Autograder Lance la correction automatisée de votre implémentation. Cliquez sur « Lancer la correction » : le système compile vos fichiers, exécute les 63 tests JUnit et affiche le score détaillé par composante. **Utilisez cet outil pour valider votre travail avant la remise.**

Barème

Implémentation — 50 pts (Autograder)

Composante	Tests	Points
`HashFrequencyTable`	16	8
`HeapTopKStrategy`	10	10
`PrefixTrie`	18	14
`KatzBackoffModel`	24	18
Total implémentation	**68**	**50**

Le score de chaque composante est proportionnel au nombre de tests réussis.

Qualité du code et des commentaires — 10 pts

Cette partie est évaluée manuellement par le correcteur. Les critères sont :

— **Lisibilité** : noms de variables clairs, structure logique du code.

— **Commentaires** : les choix algorithmiques non triviaux sont expliqués.

— **Concision** : absence de code redondant ou inutile.

— **Respect des invariants** : cohérence des champs internes après chaque opération.

Rapport — 40 pts

Question	Points
Q1 — Invariants des structures de données	12
Q2 — Complexité asymptotique de l'entraînement	12
Q3 — Complexité asymptotique des requêtes	10
Q4 — Comparaison d'implémentations alternatives	6
Total rapport	**40**

Total : 100 pts

Bonus

+5 % sur la note finale si le rapport est remis **imprimé** au cours (en plus de la remise numérique sur Studium). Le bonus s'applique à la note totale obtenue sur 100, plafonnée à 100.

Instructions de remise

À remettre sur Studium

Une archive **.zip** ou **.tar.gz** contenant :

1. Les quatre fichiers d'implémentation depuis `src/main/java/ca/udem/ift2015/autocompleter/stud`
   - `HashFrequencyTable.java`
   - `HeapTopKStrategy.java`
   - `PrefixTrie.java`
   - `KatzBackoffModel.java`
2. Le rapport complété en PDF, nommé :
   `rapport_prenom1Nom1_prenom2Nom2.pdf`
   Exemple : `rapport_aliceDupont_bobMartin.pdf`

Critères de conformité

- Le projet doit compiler sans erreur : `mvn compile`.
- Seuls les fichiers du dossier `student/` doivent avoir été modifiés.
- Aucune dépendance externe ne doit être ajoutée au `pom.xml`.
- Le rapport doit être remis au format PDF uniquement.

Remarque

L'Autograder de l'interface graphique (`mvn javafx:run`) vous permet de vérifier votre score avant la remise. Le score affiché correspond exactement à la notation de l'implémentation (50 pts).